Basics der Versionsverwaltung mit Git (lokale Repos)

TL;DR
Änderungen an Dateien (in der Workingcopy) werden mit `git add` zum "Staging" (Index) hinzugefügt. Dies ist eine Art Sammelbereich für Änderungen, die mit dem nächsten Commit in das Repository überführt werden. Neue (bisher nicht versionierte Dateien) müssen ebenfalls zunächst mit `git add` zum Staging hinzugefügt werden.
Änderungen kann man mit `git log` betrachten, dabei erhält man u.a. eine Liste der Commits und der jeweiligen Commmit-Messages.
Mit `git diff` kann man gezielt Änderungen zwischen Commits oder Branches betrachten.
Mit `git tag` kann man bestimmte Commits mit einem "Stempel" (zusätzlicher Name) versehen, um diese leichter finden zu können.
Wichtig sind die Commit-Messages: Diese sollten eine kurze Zusammenfassung haben, die **aktiv** formuliert wird (was ändert dieser Commit: "Formatiere den Java-Code entsprechend Style"; nicht aber "Java-Code nach Style formatiert"). Falls der Kommentar länger sein soll, folgt eine Leerzeile auf die erste Zeile (Zusammenfassung) und danach ein Block mit der längeren Erklärung.

Videos

- VL Git Basics
- Demo New Files
- Demo Arbeitsablauf: Datei ändern - stagen - committen
- Demo Amend
- Demo Log
- Demo Diff
- Demo Tag

Unterlagen

- PDF-Version

Versionsverwaltung mit Git: Typische Arbeitsschritte

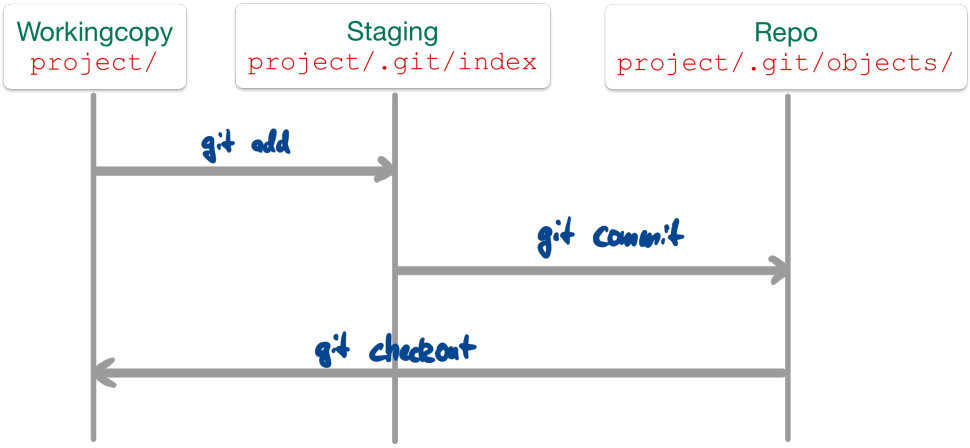
1. Repository anlegen (oder clonen)

2. Dateien neu erstellen (und löschen, umbenennen, verschieben)
3. Änderungen einpflegen ("committen")
4. Änderungen und Logs betrachten
5. Änderungen rückgängig machen
6. Projektstand markieren ("taggen")

7. Entwicklungszweige anlegen ("branchen")
8. Entwicklungszweige zusammenführen ("mergen")

9. Änderungen verteilen (verteiltes Arbeiten, Workflows)

Dateien unter Versionskontrolle stellen



1. `git add .` (oder `git add <file>`)

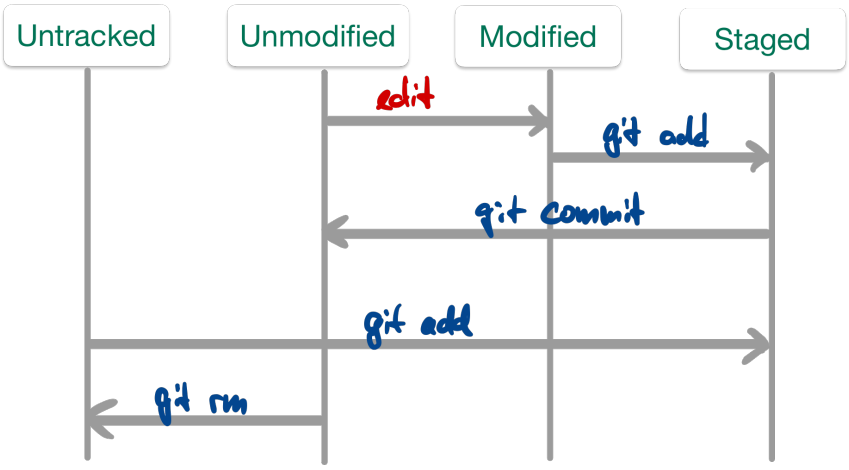
   => Stellt alle Dateien (bzw. die Datei `<file>`) im aktuellen Verzeichnis unter Versionskontrolle

2. `git commit`

   => Fügt die Dateien dem Repository hinzu

Abfrage mit `git status`

Änderungen einpflegen



- Abfrage mit: `git status`
- "Staging" von modifizierten Dateien: `git add <file>`

- Committen der Änderungen im Stage: `git commit`

Anmerkung: Alternativ auch mit `git commit -m "Kommentar"`, um das Öffnen des Editors zu vermeiden … geht einfach schneller ;)

Das "staging area" stellt eine Art Zwischenebene zwischen Working Copy und Repository dar: Die Änderungen sind temporär "gesichert", aber noch nicht endgültig im Repository eingepflegt ("committed").

Man kann den Stage dazu nutzen, um Änderungen an einzelnen Dateien zu sammeln und diese dann (in einem Commit) gemeinsam einzuchecken.

Man kann den Stage in der Wirkung umgehen, indem man alle in der Working Copy vorliegenden Änderungen per `git commit -a -m "Kommentar"` eincheckt. Der Schalter "`-a`" nimmt alle vorliegenden Änderungen an **bereits versionierten** Dateien, fügt diese dem Stage hinzu und führt dann den Commit durch. Das ist das von SVN bekannte Verhalten. Achtung: Nicht versionierte Dateien bleiben dabei außen vor!

Letzten Commit ergänzen

- `git commit --amend -m "Eigentlich wollte ich das so sagen"`

  Wenn keine Änderungen im Stage sind, wird so die letzte Commit-Message geändert.

- `git add <file>; git commit --amend`

  Damit können vergessene Änderungen an der Datei `<file>` zusätzlich im letzten Commit aufgezeichnet werden.

  In beiden Fällen ändert sich die Commit-ID!

Weitere Datei-Operationen: hinzufügen, umbenennen, löschen

- Neue (unversionierte) Dateien und Änderungen an versionierten Dateien zum Staging hinzufügen: `git add <file>`
- Löschen von Dateien (Repo+Workingcopy): `git rm <file>`
- Löschen von Dateien (nur Repo): `git rm --cached <file>`
- Verschieben/Umbenennen: `git mv <fileAlt> <fileNeu>`

Aus Sicht von Git sind zunächst alle Dateien "untracked", d.h. stehen nicht unter Versionskontrolle.

Mit `git add <file>` (und `git commit`) werden Dateien in den Index (den Staging-Bereich, d.h. nach dem Commit letztlich in das Repository) aufgenommen. Danach stehen sie unter "Beobachtung" (Versionskontrolle). So lange, wie eine Datei identisch zur Version im Repository ist, gilt sie als unverändert ("unmodified"). Eine Änderung führt entsprechend zum Zustand "modified", und ein `git add <file>` speichert die Änderungen im Stage. Ein Commit überführt die im Stage vorgemerkte Änderung in das Repo, d.h. die Datei gilt wieder als "unmodified".

Wenn eine Datei nicht weiter versioniert werden soll, kann sie aus dem Repo entfernt werden. Dies kann mit `git rm <file>` geschehen, wobei die Datei auch aus der Workingcopy gelöscht wird. Wenn die Datei erhalten

bleiben soll, aber nicht versioniert werden soll (also als "untracked" markiert werden soll), dann muss sie mit `git rm --cached <file>` aus der Versionskontrolle gelöscht werden. Achtung: Die Datei ist dann nur ab dem aktuellen Commit gelöscht, d.h. frühere Revisionen enthalten die Datei noch!

Wenn eine Datei umbenannt werden soll, geht das mit `git mv <fileAlt> <fileNeu>`. Letztlich ist dies nur eine Abkürzung für die Folge `git rm --cached <fileAlt>`, manuelles Umbenennen der Datei in der Workingcopy und `git add <fileNeu>`.

Commits betrachten

- Liste aller Commits: `git log`
  - `git log -<n>` oder `git log --since="3 days ago"` Meldungen eingrenzen ...
  - `git log --stat` Statistik ...
  - `git log --author="pattern"` Commits eines Autors
  - `git log <file>` Änderungen einer Datei

- Inhalt eines Commits: `git show`

Änderungen und Logs betrachten

- `git diff [<file>]`

  Änderungen zwischen Workingcopy und letztem Commit (ohne Stage)

  Das "staging area" wird beim Diff von Git behandelt, als wären die dort hinzugefügten Änderungen bereits eingecheckt (genauer: als letzter Commit im aktuellen Branch im Repo vorhanden). D.h. wenn Änderungen in einer Datei mittels `git add <datei>` dem Stage hinzugefügt wurden, zeigt `git diff <datei>` keine Änderungen an!

- `git diff commitA commitB`

  Änderungen zwischen Commits

- Blame: `git blame <file>`

  Wer hat was wann gemacht?

Dateien ignorieren: *.gitignore*

- Nicht alle Dateien gehören ins Repo:
  - generierte Dateien: `.class`
  - temporäre Dateien

- Datei `.gitignore` anlegen und committen
  - Wirkt auch für Unterordner
  - Inhalt: Reguläre Ausdrücke für zu ignorierende Dateien und Ordner

```
# Compiled source #
*.class
*.o
*.so

# Packages #
*.zip

# All directories and files in a directory #
bin/**/*
```

man 5 gitignore (https://linux.die.net/man/5/gitignore)

Zeitmaschine

- Änderungen in Workingcopy rückgängig machen
  - Änderungen nicht in Stage: `git checkout <file>` oder `git restore <file>`
  - Änderungen in Stage: `git reset HEAD <file>` oder `git restore --staged <file>`

  => Hinweise von `git status` beachten!

- Datei aus altem Stand holen:
  - `git checkout <commit> <file>`, oder
  - `git restore --source <commit> <file>`
- Commit verwerfen, Geschichte neu: `git revert <commit>`

Hinweis: In den neueren Versionen von Git ist der Befehl `git restore` hinzugekommen, mit dem Änderungen rückgängig gemacht werden können. Der bisherige Befehl `git checkout` steht immer noch zur Verfügung und bietet über `git restore` hinaus weitere Anwendungsmöglichkeiten.

- Stempel (Tag) vergeben: `git tag <tagname> <commit>`
- Tags anzeigen: `git tag` und `git show <tagname>`

Wann und wie committen?

Jeder Commit stellt einen Rücksetzpunkt dar!

Typische Regeln:

- Kleinere “Häppchen” einchecken: ein Feature oder Task (das nennt man auch *atomic commit*: das kleinste Set an Änderungen, die gemeinsam Sinn machen und die ggf. gemeinsam zurückgesetzt werden können)
- Logisch zusammenhängende Änderungen gemeinsam einchecken
- Projekt muss nach Commit compilierbar sein
- Projekt sollte nach Commit lauffähig sein

Ein Commit sollte in sich geschlossen sein, d.h. die kleinste Menge an Änderungen enthalten, die gemeinsam einen Sinn ergeben und die (bei Bedarf) gemeinsam zurückgesetzt oder verschoben werden können. Das nennt man auch **atomic commit**.

Wenn Sie versuchen, die Änderungen in Ihrem Commit zu beschreiben (siehe nächste Folie “Commit-Messages”), dann werden Sie einen *atomic commit* mit einem kurzen Satz (natürlich im Imperativ!) beschreiben können. Wenn Sie mehr Text brauchen, haben Sie wahrscheinlich keinen *atomic commit* mehr vor sich.

Lesen Sie dazu auch How atomic Git commits dramatically increased my productivity - and will increase yours too.

Schreiben von Commit-Messages: WARUM?!

Schauen Sie sich einmal einen Screenshot eines `git log --oneline 61e48f0..e2c8076` im Dungeon-CampusMinden/Dungeon an:

```
e2c8076 [Toolchain] Re-rename groupID (#374)
a991cf3 Repository umbenennen (#370)
8a3f213 Fix Typo in graphics Template (#360)
4cd9463 Fix für Issue 357 Pathfinding Probleme mit ID 0  (#359)
a21cd66 Merge frame methods (#353)
719836d [GITHUB] New Issue-Templates (#336)
20caad9 Update: github_ci.yml, cache gradle wrapper (#350)
0ed33af Add dungeon-starter to core (#348)
8c73ab2 [GENERATOR] Perlin Noise #332 (#334)
9c2c611 [FEATURE] Refactor Level (#329)
144d7b4 Update gson version and add dependencies checker (#344)
a75d101 [ENHANCEMENT] Add List for AbstractController (#337)
8170ade [BUG] add Javadoc for beginFrame/endFrame/setup (#339)
4348a13 Remove Xms and Xmx options (#330)
0b18d69 rename Tile.texturePath to Tile.texture (#322)
46530b6 Layer system 5 (#300)
d5c20f5 rename getTexture in getTexturePath (#312)
cb24ce5 Levelgenerierung: Remove Wildcards and Replacement (#310)
763c02e Remove version number (master, 5.x) (#301)
d73aa0c Door bug (#298)
637acbe Pausefunktion für Gameloop (#299)
1a9326c Update workflows and buildscripts (develop) (#293)
2ccce6d konstante Laufzeit der getTileAt(coordinate) Methode in Level  (#266)
0b565d4 Better start and exit (#258)
```

Nun stellen Sie sich vor, Sie sind auf der Suche nach Informationen, suchen einen bestimmten Commit oder wollen eine bestimmte Änderung finden …

Wenn man das genauer analysiert, dann stören bestimmte Dinge:

- Mischung aus Deutsch und Englisch

- “Vor-sich-hin-Murmeln”: “Layer system 5”
- Teileweise werden Tags genutzt wie `[BUG]`, aber nicht durchgängig
- Mischung zwischen verschiedenen Formen: “Repo umbenennen”, “Benenne Repo um”, “Repo umbenannt”
- Unterschiedliche Groß- und Kleinschreibung
- Sehr unterschiedlich lange Zeilen/Kommentare

Das Beachten einheitlicher Regeln ist enorm wichtig!

Leider sagt sich das so leicht - in der Praxis macht man es dann doch schnell wieder unsauber. Dennoch, auch im Dungeon-Repo gibt es einen positiven Trend (`git log --oneline 8039d6c..7f49e89`):

```
7f49e89 [GAME] add better level size generation options (#493)
c64f7c1 [GAME] add fireball textures (#477)
f645815 [GAME] remove old logo files, use new logo (#478)
3f1f2c9 new logo with the red cat in the dimensions 35x35, 64x64 and 128x128 (#476)
c6f0fc1 [Game]  Schatztruhe (#463)
803e2d0 [GAME] Change Exit/StartTile (#459)
8483714 [Game] Fix Exception in Configuration on Game Startup (#475)
cb8017f [GAME] Erweiterung des Collision checks um onEnter und onLeave (#452)
382e541 [GAME] Add fight skill (#469)
9c429af [GAME] SkillSystem (#395)
6d76e79 [GAME] add Interaktionssystem & Interaktionscomponent (#389)
a90dfe6 [GAME] add Skill Cool Down (#394)
e08c3b3 [GAME] add  Basis Logging (#455)
7290999 [GAME] fix getRandomTile mögliche Endlosschleife (#458)
13f460d [GAME] Items (Active & Passive) + Stats (#450)
5f0459f [GAME] onDeath drop Loot (#396)
03e1ff0 [GAME] Funktionales Handling von Optionals (#400)
4bb8b9b [GAME] Configuration File (#443)
2e0caaf [GAME] Inventory Component (#388)
195c686 [GAME] add XPComponent & -System
3d342bb [GAME] Merge `pm-dungeon` and `game` Projects and create `api`Project (#401)
64a0fce [GAME] Build w/ local code, i.e. do NOT use Maven Central, and remove duplicated assets (#399)
5c11d7d [GAME] IIdle: PatrouilleWalk (#352)
d0da90f [GAME] add ECS Dokumentation (#211)
```

Typische Regeln und Konventionen tauchen überall auf, beispielsweise in Chacon und Straub (2014) oder bei Tim Pope (siehe nächstes Beispiel) oder bei “How to Write a Git Commit Message”.

```
Short (50 chars or less) summary of changes

More detailed explanatory text, if necessary.  Wrap it to about
72 characters or so.  In some contexts, the first line is treated
as the subject of an email and the rest of the text as the body.
The blank line separating the summary from the body is critical
(unless you omit the body entirely); tools like rebase can get
confused if you run the two together.

Further paragraphs come after blank lines.

 - Bullet points are okay, too
 - Typically a hyphen or asterisk is used for the bullet, preceded
   by a single space, with blank lines in between, but conventions
   vary here
```

Quelle: “A Note About Git Commit Messages” by Tim Pope on tbaggery.com

Denken Sie sich die Commit-Message als E-Mail an einen zukünftigen Entwickler, der das in fünf Jahren liest!

Vom Aufbau her hat eine E-Mail auch eine Summary und dann den eigentlichen Inhalt … Erklären Sie das

“WARUM” der Änderung! (Das “WER”, “WAS”, “WANN” wird bereits automatisch von Git aufgezeichnet . . .)

Lesen (und beachten) Sie unbedingt auch “How to Write a Git Commit Message”!

Ausflug “Conventional Commits”

Die Commit-Messages dienen vor allem der Dokumentation und werden von Entwicklern gelesen.

Wenn man die Messages ein wenig stärker formalisieren würde, dann könnte man diese aber auch mit Tools verarbeiten und beispielsweise automatisiert Changelogs oder Release-Texte verfassen!

Betrachten Sie einmal das Projekt ConventionalCommits.org. Dies ist ein solcher Versuch, die Commit-Messages (a) einheitlicher und lesbarer zu gestalten und (b) auch eine Tool-gestützte Auswertung zu erlauben.

Das Projekt schlägt als Erweitung der üblichen Regeln zum Formatieren von Commit-Messages vor, dass in der ersten Zeile der *Summary* noch eine Abkürzung für die in diesem Commit erfolgte Änderung (Bug-Fix, neues Feature, . . .) vorangestellt wird. Dieser Abkürzung kann in Klammern noch der Scope der Änderung hinzugefügt werden, beispielsweise den Bereich im Projekt, der von diesem Commit berührt wird. Wenn es eine *breaking change* ist, also alter Code nach dieser Änderung sich anders verhält oder vielleicht sogar nicht mehr kompiliert, wird noch ein “!” hinter dem Typ der Änderung ergänzt.

Beispiel: Stellen Sie sich vor, im Dungeon-Projekt wurde ein neues Verhalten hinzugefügt.

1. Normalerweise hätten Sie vielleicht diese Message geschrieben (angepasste Version aus Dungeon-CampusMinden/Dungeon/pull/469):

```
add fight skill

-   `DamageProjectileSkill` creates a new entity which causes `HealthDamage` when hitting another entity
-   `FireballSkill` is a more concrete implementation of this
-   Melee skills can be created with `DamageProjectileSkill` using a customised range
    -   Example: the `FireballSkill` has a range of 10, a melee would have a considerably smaller range

fixes #24
fixes #126
fixes #224
```

2. Mit ConventionalCommits.org könnte das dann so aussehen:

```
feat: add fight skill

-   `DamageProjectileSkill` creates a new entity which causes `HealthDamage` when hitting another entity
-   `FireballSkill` is a more concrete implementation of this
-   Melee skills can be created with `DamageProjectileSkill` using a customised range
    -   Example: the `FireballSkill` has a range of 10, a melee would have a considerably smaller range

```

```
fixes #24
fixes #126
fixes #224
```

Da es sich um ein neues Feature handelt, wurde der Summary in der ersten Zeile ein `feat:` vorangestellt.

Die zu verwendenden Typen/Abkürzungen sind im Prinzip frei definierbar. Das Projekt ConventionalCommits.org schlägt eine Reihe von Abkürzungen vor. Auf diese Weise sollen in möglichst allen Projekten, die Conventional Commits nutzen, die selben Abkürzungen/Typen eingesetzt werden und so eine Tool-gestützte Auswertung möglich werden.

3. Oder zusätzlich mit dem Scope der Änderung:

```
feat(game): add fight skill

-   `DamageProjectileSkill` creates a new entity which causes `HealthDamage` when hitting another entity
-   `FireballSkill` is a more concrete implementation of this
-   Melee skills can be created with `DamageProjectileSkill` using a customised range
    -   Example: the `FireballSkill` has a range of 10, a melee would have a considerably smaller range

fixes #24
fixes #126
fixes #224
```

Der Typ `feat` wurde hier noch ergänzt um einen frei definierbaren Identifier für den Projektbereich. Dieser wird in Klammern direkt hinter den Typ notiert (hier `feat(game):`).

Im Beispiel habe ich als Bereich “game” genommen, weil die Änderung sich auf den Game-Aspekt des Projekts bezieht. Im konkreten Projekt wären andere Bereiche eventuell “dsl” (für die im Projekt entwickelte Programmiersprache plus Interpreter) und “blockly” (für die Integration von Google Blockly zur Programmierung des Dungeons mit LowCode-Ansätzen). Das ist aber letztlich vom Projekt abhängig und weitestgehend Geschmackssache.

4. Oder zusätzlich noch als Auszeichnung “breaking change” (hier mit *scope*, geht aber auch ohne *scope*):

```
feat(game)!: add fight skill

-   `DamageProjectileSkill` creates a new entity which causes `HealthDamage` when hitting another entity
-   `FireballSkill` is a more concrete implementation of this
-   Melee skills can be created with `DamageProjectileSkill` using a customised range
    -   Example: the `FireballSkill` has a range of 10, a melee would have a considerably smaller range

fixes #24
fixes #126
fixes #224
```

Angenommen, das neue Feature muss in der API etwas ändern, so dass existierender Code nun nicht mehr funktionieren würde. Dies wird mit dem extra Ausrufezeichen hinter dem Typ/Scope kenntlich gemacht

(hier `feat(game)!:`).

Zusätzlich kann man einen "Footer" in die Message einbauen, also eine extra Zeile am Ende, die mit dem String "BREAKING CHANGE:" eingeleitet wird. (vgl. Conventional Commits > Examples)

Es gibt noch viele weitere Initiativen, Commit-Messages lesbarer zu gestalten und zu vereinheitlichen. Schauen Sie sich beispielsweise einmal gitmoji.dev an. (*Mit einem Einsatz in einem professionellen Umfeld wäre ich hier aber sehr … vorsichtig.*)

Wrap-Up

- Änderungen einpflegen zweistufig (`add`, `commit`)
- Status der Workingcopy mit `status` ansehen
- Logmeldungen mit `log` ansehen
- Änderungen auf einem File mit `diff` bzw. `blame` ansehen
- Projektstand markieren mit `tag`
- Ignorieren von Dateien/Ordnern: Datei `.gitignore`

Zum Nachlesen

- Chacon und Straub (2014, Kap. 2)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Ich kann Dateien zur Versionskontrolle hinzufügen bzw. aus der Versionierung löschen
- k3: Ich kann Änderungen (geänderte Dateien) zum Staging hinzufügen und committen
- k3: Ich kann Unterschiede zwischen Commits herausfinden und mir die Historie des Repos anschauen
- k3: Ich kann gezielt Dateien und Ordner von der Versionierung ausnehmen (ignorieren)

Challenges
Versionierung 101

1. Legen Sie ein Repository an.
2. Fügen Sie Dateien dem Verzeichnis hinzu und stellen Sie *einige* davon unter Versionskontrolle.
3. Ändern Sie eine Datei und versionieren Sie die Änderung.
4. Was ist der Unterschied zwischen "`git add .; git commit`" und "`git commit -a`"?
5. Wie finden Sie heraus, welche Dateien geändert wurden?
6. Entfernen Sie eine Datei aus der Versionskontrolle, aber nicht aus dem Verzeichnis!
7. Entfernen Sie eine Datei komplett (Versionskontrolle und Verzeichnis).
8. Ändern Sie eine Datei und betrachten die Unterschiede zum letzten Commit.
9. Fügen Sie eine geänderte Datei zum Index hinzu. Was erhalten Sie bei `git diff <datei>`?
10. Wie können Sie einen früheren Stand einer Datei wiederherstellen? Wie finden Sie überhaupt den Stand?
11. Legen Sie sich ein Java-Projekt in Ihrer IDE an an. Stellen Sie dieses Projekt unter Git-Versionskontrolle. Führen Sie die vorigen Schritte mit Ihrer IDE durch.

Quellen

Atlassian Pty Ltd. 2022. „Become a Git Guru." 2022. https://www.atlassian.com/git/tutorials.
Chacon, S., und B. Straub. 2014. *Pro Git*. 2. Aufl. Apress. https://git-scm.com/book/en/v2.
Github Inc. 2022. „Git Cheat Sheets". 2022. https://training.github.com/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- "A Note About Git Commit Messages" by Tim Pope on tbaggery.com

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)